

OK so give final components list

Here is your **final hardware components list** for moving from the **prototype phase to a production-ready AI defect detection system**, assuming:

- You already have a **laptop (Windows)**
- Prototype will use basic, low-cost components
- Upgrade path is built-in toward an industrial solution

 Final Component List (for Prototype & Scalable Transition)

Category	Component	Purpose	Approx. Cost (INR)
 Controller	Laptop (Already owned)	AI processing, image analysis	₹0
	(Optional) ESP32 Dev Board	For GPIO sensor input/output control	₹500 – ₹800
 Imaging	USB Webcam (1080p HD)	Captures product images for AI inference	₹600 – ₹1,000
	(Upgrade) Industrial USB3 Camera (e.g., 5MP)	Future-proof high-quality inspection	₹15,000 – ₹30,000
 Lighting	LED Strip / Ring Light	Uniform lighting for visibility	₹300 – ₹700
 Sensors	IR/Photoelectric Sensor (e.g., TCRT5000 or E18-D80NK)	Detect object presence on surface	₹150 – ₹300
 Actuators	Servo Motor (SG90 or MG996R)	Pushes defective part to bin (optional)	₹200 – ₹500
	Buzzer / LED Indicators	Audio/visual status indicators	₹30 – ₹80

Category	Component	Purpose	Approx. Cost (INR)
💡 Power	SMPS 5V/12V 2A Adapter	Power for servo, sensors, ESP32	₹250 – ₹400
🔧 Wiring & Mounts	Jumper Wires, Breadboard, PCB	Connection + basic circuit	₹200 – ₹300
	Acrylic / Wood Mount Platform	Mount for webcam, sensor, and product	₹300 – ₹800

📋 Total Approximate Cost (Prototype Setup)

- ✅ With ESP32 + USB Webcam + Servo: ₹2,500 – ₹4,000
- ✅ Without ESP32 (Laptop-only vision): ₹2,000 – ₹3,000
- ✅ Future-ready with USB3 Camera (Optional): Add ₹15k+

🔧 Optional Scalable Components (For Future Industrial Use)

Component	Purpose	Cost (INR)
NVIDIA Jetson Orin Nano (8GB)	AI edge inference (replaces laptop)	₹42,000
Telecentric Lens + Lighting Kit	Precise silhouette imaging	₹80,000 – ₹1.2L
Delta PLC + IO module	Industrial actuator control	₹10,000 – ₹20,000
Conveyor Setup	Inline inspection system	₹10,000 – ₹25,000

Let me know if you'd like a **purchasing link list**, **circuit diagram**, or **software integration guide** tailored to this setup.

📄 👍 💬 ↗️ ↺️ ⋮

now make one month project development plan phase wise fro 4 members in detalied way and what to verify

after one phase completes.

Here's a **detailed one-month (4-week) project development plan** for your **4-member team**, tailored to the low-cost prototype system using your laptop + USB webcam + sensors + optional ESP32.

One-Month Project Plan (Prototype Phase)

Team Structure Suggestion:

Member	Role
A	Software Developer (AI/ML + Python + UI)
B	Hardware Integrator (Wiring, Sensor setup)
C	Data Engineer (Image collection/labeling)
D	Project Lead (Testing + Integration + Report)

Week 1 – Planning & Basic Setup

Goals:

- Finalize use-case, environment & basic component wiring
- Set up imaging and sensing hardware
- Confirm trigger system works (e.g., sensor → capture image)

Tasks:

- Collect sample defective + non-defective parts (Client-side)
- Mount webcam, sensor, and lighting on mock conveyor/table
- Wire ESP32 (if used) with sensor & buzzer
- Laptop: Install Python, OpenCV, necessary libraries

Verification:

- Sensor accurately detects objects
 - Camera captures clear images on trigger
 - ESP32 (optional): responds to triggers and controls buzzer/servo
-

📌 Week 2 – Image Dataset & AI Model Training

✅ Goals:

- Build labeled dataset from real parts
- Train a basic image classifier or object detection model

🔧 Tasks:

- Capture 300–500 images of different bolt/nut samples
- Label images using Roboflow/LabelImg
- Train a YOLOv8 (or MobileNet if classifier) using Google Colab or laptop
- Store trained weights for real-time inference

🔍 Verification:

- Model achieves $\geq 85\%$ classification accuracy (train/test split)
 - Images are properly labeled (3 categories: good/acceptable/reject)
 - Can run basic inference on test images from webcam
-

📌 Week 3 – Integration & Control Logic

✅ Goals:

- Integrate AI model with real-time webcam feed
- Trigger classification on sensor signal
- Actuate buzzer/servo based on result

🔧 Tasks:

- Write Python script: sensor → capture → predict → sort
- Use thresholds to classify predictions (confidence-based)
- Control GPIO pins (via ESP32 or USB-serial) to trigger actuator
- Add basic logging of classified results (CSV/SQLite)

🔍 Verification:

- On product trigger, camera captures and classifies in ≤ 2 seconds
 - Buzzer sounds / servo moves appropriately
 - Logs classification with timestamp
-

📌 Week 4 – Testing, Refinement & Presentation

✅ Goals:

- Test complete workflow with 30–50 new samples
- Debug edge cases (false positives/negatives)

- Prepare short demo for client

Tasks:

- Run full system for 50–100 trials; collect accuracy stats
- Improve lighting or preprocessing if needed
- Prepare small UI (Tkinter/web-based) to show live output
- Record video and screenshots of working prototype

Verification:

- Achieve at least 90% classification accuracy on unseen samples
- Stable system run for 10+ minutes without crash
- Ready with short demo and documentation

Deliverables at End of Month:

-  Working low-cost prototype system
-  Real-time classification logic
-  Basic actuator response to defect
-  Dataset of images and labeled annotations
-  Short demo video
-  Technical documentation (PDF or PPT for sponsor)

Let me know if you want this plan formatted as a PDF or visual Gantt chart.

